

Базовий опис:

База даних призначена для Telegram-бота, який використовується для продажу цифрових товарів, обробки замовлень, проведення оплат, застосування промокодів, ведення реферальної системи, публікації інформаційних повідомлень і збереження історії виконання команд користувачами.

Система дозволяє зберігати інформацію про користувачів, товари, замовлення, платежі, промокоди, реферальні зв'язки, публікації та команди бота.

Таблиця bot_user

Таблиця bot_user зберігає інформацію про користувачів Telegram-бота.

Поля таблиці:

user_id - первинний ключ;

telegram_id - Telegram ID користувача;

username - ім'я користувача в Telegram;

role - роль користувача;

status - статус користувача;

registered_at - дата реєстрації;

balance - баланс користувача.

Обмеження:

user_id є первинним ключем;

telegram_id та username є унікальними;

role може мати значення user або admin;

status може мати значення active або blocked;

balance не може бути від'ємним.

Таблиця product

Таблиця product зберігає інформацію про цифрові товари.

Поля таблиці:

product_id - первинний ключ;

name - назва товару;

product_type - тип товару;

price - ціна товару;

is_active - ознака активності товару.

Обмеження:

product_id є первинним ключем;

price не може бути від'ємною;

основні поля мають обмеження NOT NULL.

Таблиця promo_code

Таблиця promo_code зберігає інформацію про промокоди.

Поля таблиці:

promo_code_id - первинний ключ;

code - текст промокоду;

discount_type - тип знижки;

discount_value - значення знижки;

starts_at - дата початку дії;

ends_at - дата завершення дії.

Обмеження:

promo_code_id є первинним ключем;

code є унікальним;

discount_type може мати значення percent або fixed;

discount_value повинен бути більшим за 0;

Таблиця bot_command

Таблиця bot_command зберігає список команд, які підтримує Telegram-бот.

Поля таблиці:

command_id - первинний ключ;

command_name - назва команди;

description - опис команди;

is_active - ознака активності команди.

Обмеження:

command_id є первинним ключем;

command_name є унікальним.

Таблиця referral

Таблиця referral зберігає інформацію про реферальні зв'язки між користувачами.

Поля таблиці:

referral_id - первинний ключ;

referrer_id - користувач, який запросив іншого користувача;

referred_id - запрошений користувач;

reward_amount - сума винагороди;

created_at - дата створення запису.

Обмеження:

referral_id є первинним ключем;

referrer_id та referred_id є зовнішніми ключами до bot_user(user_id);

користувач не може запросити самого себе;

reward_amount не може бути від'ємною.

Таблиця publication

Таблиця publication зберігає інформаційні публікації бота.

Поля таблиці:

publication_id - первинний ключ;

author_id - автор публікації;

title - заголовок;

content - зміст публікації;

tag_list - список тегів;

created_at - дата створення.

Обмеження:

publication_id є первинним ключем;

author_id є зовнішнім ключем до bot_user(user_id);

title та content є обов'язковими полями.

Таблиця bot_order

Таблиця bot_order зберігає інформацію про замовлення користувачів.

Поля таблиці:

order_id - первинний ключ;

user_id - користувач, який створив замовлення;

promo_code_id - застосований промокод;

status - статус замовлення;

created_at - дата створення;

total_amount - загальна сума замовлення.

Обмеження:

order_id є первинним ключем;

user_id є зовнішнім ключем до bot_user(user_id);

promo_code_id є зовнішнім ключем до promo_code(promo_code_id);

Таблиця order_item

Таблиця order_item зберігає товари, які входять до замовлення.

Поля таблиці:

order_item_id - первинний ключ;

order_id - замовлення;

product_id - товар;

quantity - кількість товару;

unit_price - ціна одиниці товару;

line_total - загальна сума позиції.

Обмеження:

order_item_id є первинним ключем;

order_id є зовнішнім ключем до bot_order(order_id);

product_id є зовнішнім ключем до product(product_id);

quantity повинна бути більшою за 0;

unit_price та line_total не можуть бути від'ємними;

пара order_id і product_id є унікальною.

Таблиця payment

Таблиця payment зберігає інформацію про оплату замовлень.

Поля таблиці:

payment_id - первинний ключ;

order_id - замовлення;

amount - сума платежу;

method - спосіб оплати;

status - статус платежу;

paid_at - дата та час оплати.

Обмеження:

payment_id є первинним ключем;

order_id є зовнішнім ключем до bot_order(order_id);

order_id є унікальним, тому одне замовлення може мати тільки один платіж;

method може мати значення card, balance, crypto, manual;

status може мати значення pending, paid, failed, refunded;

amount не може бути від'ємною.

Таблиця command_execution

Таблиця command_execution зберігає історію виконання команд користувачами.

Поля таблиці:

execution_id - первинний ключ;

user_id - користувач;

command_id - команда;

executed_at - дата та час виконання;

result_status - результат виконання.

Обмеження:

execution_id є первинним ключем;

user_id є зовнішнім ключем до bot_user(user_id);

command_id є зовнішнім ключем до bot_command(command_id);

result_status може мати значення success або error.

Зв'язки між таблицями

У базі даних реалізовано такі зв'язки:

Один користувач може створити багато замовлень.

Зв'язок реалізовано через `bot_order.user_id`.

Один користувач може створити багато публікацій.

Зв'язок реалізовано через `publication.author_id`.

Один користувач може запросити багато інших користувачів.

Зв'язок реалізовано через таблицю `referral`.

Одне замовлення може містити багато товарів, а один товар може входити до багатьох замовлень.

Зв'язок реалізовано через таблицю `order_item`.

Одне замовлення може мати один платіж.

Зв'язок реалізовано через `payment.order_id`.

Один промокод може використовуватися в багатьох замовленнях.

Зв'язок реалізовано через `bot_order.promo_code_id`.

Зв'язок реалізовано через таблицю `command_execution`.

Тестові дані:

Для перевірки роботи схеми були додані тестові записи в усі таблиці:

`bot_user` - 5 записів;

`product` - 5 записів;

`promo_code` - 3 записи;

`bot_command` - 5 записів;

`referral` - 3 записи;

`publication` - 3 записи;

`bot_order` - 5 записів;

`order_item` - 5 записів;

`payment` - 5 записів;

`command_execution` - 5 записів.

Перевірка роботи

Для перевірки заповнення таблиць були виконані SQL-запити:

```
SELECT * FROM bot_user;
```

```
SELECT * FROM product;
```

```
SELECT * FROM bot_order;
```

```
SELECT * FROM payment;
```

Також було виконано запит для перевірки кількості записів у кожній таблиці:

```
SELECT 'bot_user' AS table_name, COUNT(*) AS rows_count FROM bot_user
```

```
UNION ALL
```

```
SELECT 'product', COUNT(*) FROM product
```

```
UNION ALL
```

```
SELECT 'promo_code', COUNT(*) FROM promo_code
```

```
UNION ALL
```

```
SELECT 'bot_command', COUNT(*) FROM bot_command
```

```
UNION ALL
```

```
SELECT 'referral', COUNT(*) FROM referral
```

```
UNION ALL
```

```
SELECT 'publication', COUNT(*) FROM publication
```

```
UNION ALL
```

```
SELECT 'bot_order', COUNT(*) FROM bot_order
```

```
UNION ALL
```

```
SELECT 'order_item', COUNT(*) FROM order_item
```

```
UNION ALL
```

```
SELECT 'payment', COUNT(*) FROM payment
```

```
UNION ALL
```

```
SELECT 'command_execution', COUNT(*) FROM command_execution;
```


Висновок:

У результаті виконання лабораторної роботи було реалізовано реляційну схему PostgreSQL для бази даних Telegram-бота. Було створено таблиці для користувачів, товарів, замовлень, платежів, промокодів, публікацій, реферальної системи та команд бота.

У таблицях були визначені первинні та зовнішні ключі, а також обмеження UNIQUE, NOT NULL, CHECK і DEFAULT. Для перевірки роботи схеми були додані тестові записи та виконані SQL-запити SELECT. Отримані результати підтвердили, що таблиці створені правильно, зв'язки між ними працюють, а дані успішно збережені в базі даних.